

Front-end code

components / AnalysisResultsView.js and FeedbackAnalyzer.js

AnalysisResultsView.js

```
import React, { useState } from 'react';
import {
  BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, Legend,
  ResponsiveContainer
} from 'recharts';
const AnalysisResultsView = ({ results }) => {
  const [activeTab, setActiveTab] = useState('summary');

  const prepareThemeChartData = () => {
    if (!results?.key_themes) return [];

    return Object.entries(results.key_themes)
      .sort((a, b) => b[1] - a[1])
      .slice(0, 10)
      .map(([theme, count]) => ({
        theme: theme.charAt(0).toUpperCase() + theme.slice(1),
        mentions: count
      }));
  };

  if (!results) return null;

  const stats = results.statistics || {};
  const summaryText = typeof results.summary === 'string' ? results.summary : '';

  return (
    <div className="analysis-results-view">
      <div className="stats-grid">
        <div className="stat-card total">
          <div className="stat-icon">  </div>
          <div className="stat-content">
            <div className="stat-value">{results.total_feedback ?? 0}</div>
            <div className="stat-label">Total Feedback</div>
          </div>
        </div>
      </div>
    </div>
  );
};
```

Source-Code

```
<div className="stat-card clusters">
  <div className="stat-icon"> 🧠 </div>
  <div className="stat-content">
    <div className="stat-value">{stats.clusters_found ?? 0}</div>
    <div className="stat-label">Clusters Found</div>
  </div>
</div>

<div className="stat-card themes">
  <div className="stat-icon"> 🎯 </div>
  <div className="stat-content">
    <div className="stat-value">{stats.themes_identified ?? 0}</div>
    <div className="stat-label">Key Themes</div>
  </div>
</div>
</div>

<div className="tabs">
  <button
    type="button"
    className={`tab ${activeTab === 'summary' ? 'active': ''}`}
    onClick={() => setActiveTab('summary')}
  >
    📄 AI Summary
  </button>
  <button
    type="button"
    className={`tab ${activeTab === 'suggestions' ? 'active': ''}`}
    onClick={() => setActiveTab('suggestions')}
  >
    💡 Recommendations
  </button>
  <button
    type="button"
    className={`tab ${activeTab === 'themes' ? 'active': ''}`}
    onClick={() => setActiveTab('themes')}
  >
    🎯 Themes Analysis
  </button>
```

Source-Code

```
<button
  type="button"
  className={`tab ${activeTab === 'clusters' ? 'active': ''}`}
  onClick={() => setActiveTab('clusters')}
>
  🔍 Cluster Details
</button>
</div>
```

```
<div className="tab-content">
  {activeTab === 'summary' && (
    <div className="summary-section">
      <h2> 📄 Comprehensive AI Analysis</h2>
      <div className="summary-content">
        {summaryText.split('\n').map((paragraph, index) => {
          if (paragraph.trim().startsWith('##')) {
            return (
              <h3 key={index} className="summary-heading">
                {paragraph.replace(/##/g, "").trim()}
              </h3>
            );
          }
          if (paragraph.trim()) {
            return (
              <p key={index} className="summary-paragraph">
                {paragraph.trim()}
              </p>
            );
          }
          return null;
        })}
      </div>
    </div>
  )}
```

```
{activeTab === 'suggestions' && (
  <div className="suggestions-section">
    <h2> 💡 AI-Generated Recommendations</h2>
    <p className="section-description">
      Specific, actionable recommendations based on customer feedback analysis
    </p>
  </div>
)}
```

Source-Code

```
</p>
<div className="suggestions-list">
  {(results.suggestions || []).map((suggestion, index) => {
    const lines = String(suggestion).split('\n').filter(line => line.trim());
    const title = lines[0] || suggestion;
    const description = lines.slice(1).join(' ') || '';

    return (
      <div key={index} className="suggestion-item-detailed">
        <div className="suggestion-header">
          <span className="suggestion-number">{index + 1}</span>
          <h3 className="suggestion-title">{title}</h3>
        </div>
        {description && (
          <p className="suggestion-description">{description}</p>
        )}
      </div>
    );
  })}
</div>
</div>
)}

{activeTab === 'themes' && (
  <div className="themes-section">
    <h2>🎯 Top Themes Identified</h2>
    <ResponsiveContainer width="100%" height={400}>
      <BarChart data={prepareThemeChartData()}>
        <CartesianGrid strokeDasharray="3 3" />
        <XAxis dataKey="theme" angle={-45} textAnchor="end" height={100} />
        <YAxis />
        <Tooltip />
        <Legend />
        <Bar dataKey="mentions" fill="#667eea" />
      </BarChart>
    </ResponsiveContainer>

    <div className="themes-list">
      <h3>📄 Theme Details</h3>
      <div className="theme-items">
        {Object.entries(results.key_themes || {})
```

Source-Code

```
.sort((a, b) => b[1] - a[1])
.map(([theme, count]) => (
  <div key={theme} className="theme-item">
    <span className="theme-name">{theme.toUpperCase()}</span>
    <span className="theme-count">{count} mentions</span>
  </div>
))}
</div>
</div>
</div>
))}

{activeTab === 'clusters' && (
  <div className="clusters-section">
    <h2>🔍 Feedback Clusters Analysis</h2>
    <p className="section-description">
      Feedback automatically grouped into {results.cluster_info?.n_clusters || 0}
      semantic clusters based on similarity.
    </p>

    {results.cluster_info?.cluster_summaries &&
      Object.keys(results.cluster_info.cluster_summaries).length > 0 ? (
        <div className="clusters-grid">
          {Object.entries(results.cluster_info.cluster_summaries).map(([clusterId,
cluster]) => (
            <div key={clusterId} className="cluster-card">
              <div className="cluster-header">
                <h3>Cluster {parseInt(clusterId, 10) + 1}</h3>
                <span className="cluster-size">{cluster.size} feedbacks</span>
              </div>
              <p className="cluster-summary">{cluster.summary}</p>
              {cluster.sample_feedback && (
                <div className="cluster-sample">
                  <strong>Sample:</strong> {cluster.sample_feedback.substring(0, 150)}...
                </div>
              )}
            </div>
          ))}
        </div>
      ) : (
        <div className="no-clusters-message">
```

Source-Code

```
<p> ⚠ No clusters found. This could happen if:</p>
<ul>
  <li>There are too few feedback items (need at least 3-4 items for clustering)</li>
  <li>The feedback items are too similar to form distinct clusters</li>
  <li>An error occurred during clustering (check backend logs)</li>
  <li>Embeddings generation failed</li>
</ul>
<p>Try analyzing more feedback data or check the backend logs for errors.</p>
</div>
  })
</div>
})
</div>
</div>
);
};
export default AnalysisResultsView;
```

FeedbackAnalyzer.js

```
import React, { useState, useEffect, useCallback } from 'react';
import axios from 'axios';
import '../styles/FeedbackAnalyzer.css';
import AnalysisResultsView from './AnalysisResultsView';

const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:8000';

/** API sends UTC; naive ISO strings are interpreted as UTC by appending Z. */
function formatSavedAtLocal(iso) {
  if (!iso) return '';
  const s = String(iso).trim();
  const hasTz = /[zZ]${[+ -]\d{2}}:?\d{2}$/.test(s);
  const d = new Date(hasTz ? s : ` ${s}Z`);
  if (Number.isNaN(d.getTime())) return s;
  return d.toLocaleString(undefined, { dateStyle: 'medium', timeStyle: 'short' });
}

function truncatePreview(text, maxLen = 140) {
  const s = (text || '').replace(/\s+/g, ' ').trim();
  if (!s) return '—';
}
```

Source-Code

```
if (s.length <= maxLen) return s;
return `${s.slice(0, maxLen - 1)}...`;
}

const FeedbackAnalyzer = () => {
  const [formData, setFormData] = useState({
    spreadsheetId: '',
    rangeName: 'Sheet1!B:B'
  });

  const [loading, setLoading] = useState(false);
  const [results, setResults] = useState(null);
  const [error, setError] = useState(null);

  const [exporting, setExporting] = useState({ pdf: false, excel: false });

  const [datasetName, setDatasetName] = useState('');
  const [saving, setSaving] = useState(false);
  const [saveMessage, setSaveMessage] = useState(null);

  const [historyList, setHistoryList] = useState([]);
  const [historyLoading, setHistoryLoading] = useState(false);
  const [deletingId, setDeletingId] = useState(null);

  const [modalOpen, setModalOpen] = useState(false);
  const [modalLoading, setModalLoading] = useState(false);
  const [modalResults, setModalResults] = useState(null);
  const [modalMeta, setModalMeta] = useState(null);

  /** home | history | export — only one main section visible */
  const [activeNav, setActiveNav] = useState('home');

  const fetchHistory = useCallback(async () => {
    setHistoryLoading(true);
    try {
      const { data } = await axios.get(`${API_URL}/api/saved-analyses`);
      setHistoryList(Array.isArray(data) ? data : []);
    } catch (err) {
      console.error('Failed to load history:', err);
    } finally {
      setHistoryLoading(false);
    }
  }, [API_URL]);
```

Source-Code

```
}  
}, []);  
  
useEffect(() => {  
  fetchHistory();  
}, [fetchHistory]);  
  
useEffect(() => {  
  window.scrollTo({ top: 0, behavior: 'smooth' });  
}, [activeNav]);  
  
const handleExport = async (format) => {  
  if (!results) return;  
  
  setExporting(prev => ({ ...prev, [format]: true }));  
  
  try {  
    const response = await axios.post(  
      `${API_URL}/api/export/${format}`,  
      {  
        analysis_data: results,  
        metadata: results.metadata || {}  
      },  
      {  
        responseType: 'blob'  
      }  
    );  
  
    const url = window.URL.createObjectURL(new Blob([response.data]));  
    const link = document.createElement('a');  
    link.href = url;  
  
    const timestamp = new Date().toISOString().slice(0, 19).replace(/:/g, '-');  
    const extension = format === 'pdf' ? 'pdf' : 'xlsx';  
    link.setAttribute('download', `feedback_analysis_${timestamp}.${extension}`);  
  
    document.body.appendChild(link);  
    link.click();  
    link.remove();  
    window.URL.revokeObjectURL(url);  
  } catch (err) {
```


Source-Code

```
    console.error(` Export error:`, err);
    setError(` Failed to export ${format.toUpperCase()}: ${err.response?.data?.error ||
err.message}` );
  } finally {
    setExporting(prev => ({ ...prev, [format]: false }));
  }
};
```

```
const handleSaveToDatabase = async () => {
  if (!results) return;
  const name = datasetName.trim();
  if (!name) {
    setSaveMessage('Please enter a dataset name.');
```

```
    return;
  }

  setSaving(true);
  setSaveMessage(null);
```

```
  try {
    await axios.post(`${API_URL}/api/saved-analyses`, {
      dataset_name: name,
      analysis_data: results
    });
    setSaveMessage('Saved successfully.');
```

```
    setDatasetName('');
    fetchHistory();
  } catch (err) {
    const msg = err.response?.data?.error || err.message || 'Save failed';
    setSaveMessage(msg);
  } finally {
    setSaving(false);
  }
};

const openSavedModal = async (id) => {
  setModalOpen(true);
  setModalLoading(true);
  setModalResults(null);
```

Source-Code

```
setModalMeta(null);

try {
  const { data } = await axios.get(` ${API_URL}/api/saved-analyses/${id}` );
  setModalMeta({
    id: data.id,
    dataset_name: data.dataset_name,
    saved_at: data.saved_at,
    api_key_masked: data.api_key_masked,
    overall_description: data.overall_description || "",
    recommendations: Array.isArray(data.recommendations) ? data.recommendations
: []
  });
  setModalResults(data.analysis_data);
} catch (err) {
  console.error(err);
  setModalMeta({ error: err.response?.data?.error || err.message });
} finally {
  setModalLoading(false);
}
};

const closeSavedModal = () => {
  setModalOpen(false);
  setModalResults(null);
  setModalMeta(null);
};

const handleDeleteSaved = async (id) => {
  if (!window.confirm('Delete this saved analysis from the database? This cannot be undone.')) {
    return;
  }
  setDeletingId(id);
  try {
    await axios.delete(` ${API_URL}/api/saved-analyses/${id}` );
    if (modalOpen && modalMeta?.id === id) {
      closeSavedModal();
    }
  }
  fetchHistory();
} catch (err) {
```

Source-Code

```
    console.error(err);
    setError(err.response?.data?.error || err.message || 'Delete failed');
  } finally {
    setDeletingId(null);
  }
};
```

```
const handleInputChange = (e) => {
  const { name, value } = e.target;
  setFormData(prev => ({
    ...prev,
    [name]: value
  }));
};
```

```
const handleSubmit = async (e) => {
  e.preventDefault();
  setLoading(true);
  setError(null);
  setResults(null);
```

```
  try {
    const response = await axios.post(`${API_URL}/api/analyze-feedback`, {
      spreadsheet_id: formData.spreadsheetId,
      range_name: formData.rangeName
    });

    setResults(response.data);
    setActiveNav('home');
  } catch (err) {
    const errorMessage = err.response?.data?.error || err.message || 'Failed to analyze feedback';
    setError(errorMessage);
    console.error('Analysis error:', err);
  } finally {
    setLoading(false);
  }
};
```

```
const exportPanel = results && results.success ? (
  <div className="export-section">
```

Source-Code

```
<h3>  Export Report</h3>
<div className="export-buttons">
  <button
    type="button"
    className="export-btn pdf-btn"
    onClick={() => handleExport('pdf')}
    disabled={exporting.pdf}
  >
    {exporting.pdf ? ' ⌚ Exporting...': '  Export PDF'}
  </button>
  <button
    type="button"
    className="export-btn excel-btn"
    onClick={() => handleExport('excel')}
    disabled={exporting.excel}
  >
    {exporting.excel ? ' ⌚ Exporting...': '  Export Excel'}
  </button>
</div>

<div className="save-to-db-block">
  <h4 className="save-to-db-title">  Save to database</h4>
  <p className="save-to-db-hint">
    Stores structured fields (date/time, dataset name, masked API key, up to 3-line
    summary, up to 5 recommendations) plus a full snapshot for charts and PDF. Uses
    MySQL when <code className="save-inline-code">MYSQL_DATABASE</code> is set
    in backend <code className="save-inline-code">.env</code>.
  </p>
  <div className="save-to-db-row">
    <input
      type="text"
      className="save-dataset-input"
      value={datasetName}
      onChange={(e) => setDatasetName(e.target.value)}
      placeholder="Dataset name (e.g. Q1 Customer Survey)"
      maxLength={512}
    />
    <button
      type="button"
      className="save-db-btn"
```

Source-Code

```
      onClick={handleSaveToDatabase}
      disabled={saving}
    >
      {saving ? ' ⌚ Saving...' : 'Save to Database'}
    </button>
  </div>
  {saveMessage && (
    <p className={` save-message ${saveMessage.includes('success')} ? 'save-
message-ok' : 'save-message-err'} `}>
      {saveMessage}
    </p>
  )}
</div>
</div>
): null;

return (
  <div className="feedback-analyzer">
    <nav className="app-navbar" aria-label="Main navigation">
      <button
        type="button"
        className={` app-nav-brand ${activeNav === 'home' ? 'active' : ''} `}
        onClick={() => setActiveNav('home')}
      >
        SMART FEEDBACK AI
      </button>
      <div className="app-nav-links">
        <button
          type="button"
          className={` app-nav-link ${activeNav === 'home' ? 'active' : ''} `}
          onClick={() => setActiveNav('home')}
        >
          Home
        </button>
        <button
          type="button"
          className={` app-nav-link ${activeNav === 'history' ? 'active' : ''} `}
          onClick={() => {
            setActiveNav('history');
            fetchHistory();
          }}
        >
```

Source-Code

```
>
  Analysis history
</button>
<button
  type="button"
  className={` app-nav-link ${activeNav === 'export' ? 'active' : ''}`}
  onClick={() => setActiveNav('export')}
>
  Export Report
</button>
</div>
</nav>

<div className="container">
  {error && (
    <div className="error-message error-banner">
      <strong> ⚠ Error:</strong> {error}
    </div>
  )}

  {activeNav === 'home' && (
    <>
      <header className="header">
        <h1> 🤖 SMART FEEDBACK AI</h1>
        <p className="subtitle">AI Analysis System</p>
      </header>

      <div className="input-section">
        <form onSubmit={handleSubmit}>
          <div className="form-group">
            <label htmlFor="spreadsheetId">
              📊 Google Sheet ID
              <span className="required">*</span>
            </label>
            <input
              type="text"
              id="spreadsheetId"
              name="spreadsheetId"
              value={formData.spreadsheetId}
              onChange={handleInputChange}
            >
```

Source-Code

```
placeholder="Enter your Google Spreadsheet ID"
required
disabled={loading}
/>
<small className="help-text">
  Find this in your Google Sheet URL:
  docs.google.com/spreadsheets/d/<strong>[YOUR_SHEET_ID]</strong>/edit
</small>
</div>
```

```
<div className="form-group">
  <label htmlFor="rangeName">
    📌 Range (Optional)
  </label>
  <input
    type="text"
    id="rangeName"
    name="rangeName"
    value={formData.rangeName}
    onChange={handleInputChange}
    placeholder="Sheet1!B:B"
    disabled={loading}
  />
  <small className="help-text">
    Default: Sheet1!B:B (Column B only - feedback text)
  </small>
</div>
```

```
<button
  type="submit"
  className="submit-button"
  disabled={loading}
>
  {loading ? (
    <>
      <span className="spinner"></span>
      Analyzing Feedback...
    </>
  ) : (
    <>
      <span>🚀</span>
    </>
  )}
```

Source-Code

```
Analyze Feedback
</>
})
</button>
</form>
</div>

{results && results.success && (
  <div className="results-section">
    <p className="nav-hint">
      Analysis ready. Use <strong>Export Report</strong> in the top bar for PDF,
      Excel, shareable link, and database save.
    </p>
    <AnalysisResultsView results={results} />
  </div>
)}
</>
})

{activeNav === 'history' && (
  <section className="history-section history-section--page" aria-
labelledby="history-heading">
    <h2 id="history-heading" className="history-heading">  Analysis history</h2>
    <p className="history-description">
      Saved rows include date/time, dataset name, masked API key, a short summary
      (up to 3 lines), and recommendations.
      Configure MySQL with <code className="history-inline-
code">MYSQL_*</code> in backend <code className="history-inline-
code">.env</code> (see <code className="history-inline-
code">.env.example</code>).
    </p>
    {historyLoading ? (
      <p className="history-loading">Loading history...</p>
    ) : historyList.length === 0 ? (
      <p className="history-empty">No saved analyses yet. Run an analysis, then
      use <strong>Export Report</strong> → Save to Database.</p>
    ) : (
      <div className="history-table-wrap">
        <table className="history-table">
          <thead>
            <tr>
```


Source-Code

```
<th>Saved at</th>
<th>Dataset name</th>
<th>API key</th>
<th>Actions</th>
</tr>
</thead>
<tbody>
{historyList.map((row) => (
  <tr key={row.id}>
    <td>{formatSavedAtLocal(row.saved_at)}</td>
    <td>{row.dataset_name}</td>
    <td><code className="history-key">{row.api_key}</code></td>
    <td className="history-actions">
      <button
        type="button"
        className="history-btn history-btn-view"
        onClick={() => openSavedModal(row.id)}
      >
        View
      </button>
      <button
        type="button"
        className="history-btn history-btn-delete"
        onClick={() => handleDeleteSaved(row.id)}
        disabled={deletingId === row.id}
      >
        {deletingId === row.id ? '...' : 'Delete'}
      </button>
    </td>
  </tr>
)}}
</tbody>
</table>
</div>
)}
</section>
)}

{activeNav === 'export' && (
  <div className="export-report-page">
    <header className="section-page-header">
```

Source-Code

```
<h2> 📄 Export Report</h2>
<p className="section-page-sub">Download PDF or Excel, create a shareable
link, or save the current analysis to the database.</p>
</header>
{!results || !results.success ? (
  <div className="export-page-empty">
    <p>No analysis loaded yet.</p>
    <button
      type="button"
      className="export-page-cta"
      onClick={() => setActiveNav('home')}
    >
      Go to Home to analyze feedback
    </button>
  </div>
) : (
  <div className="results-section results-section--export-only">
    {exportPanel}
  </div>
)}
</div>
)}
</div>

{modalOpen && (
  <div
    className="saved-modal-overlay"
    role="presentation"
    onClick={closeSavedModal}
  >
    <div
      className="saved-modal"
      role="dialog"
      aria-modal="true"
      aria-labelledby="saved-modal-title"
      onClick={(e) => e.stopPropagation()}
    >
      <div className="saved-modal-header">
        <h2 id="saved-modal-title">
          {modalMeta?.dataset_name || 'Saved analysis'}
        </h2>
      </div>
    </div>
  </div>
)}
```

Source-Code

```
        <button type="button" className="saved-modal-close"
onClick={closeSavedModal} aria-label="Close">
            x
        </button>
    </div>

    {modalLoading && <p className="saved-modal-loading">Loading...</p>}

    {modalMeta?.error && (
        <div className="error-message">
            <strong> ⚠ Error:</strong> {modalMeta.error}
        </div>
    )}

    {!modalLoading && modalMeta && !modalMeta.error && (
        <div className="saved-modal-meta">
            <span><strong>Saved:</strong>
{formatSavedAtLocal(modalMeta.saved_at)}</span>
            <span><strong>Dataset:</strong> {modalMeta.dataset_name}</span>
        </div>
    )}

    {!modalLoading && modalResults && modalResults.success && (
        <div className="saved-modal-body">
            <h4 className="saved-modal-full-heading">Full analysis (dashboard
view)</h4>
            <AnalysisResultsView results={modalResults} />
        </div>
    )}
</div>
</div>
    )}
</div>
);
};

export default FeedbackAnalyzer;
```

backend - code

backend/app /**services**

__init__.py

```
"""
Services package initialization
"""

from .sheets_service import GoogleSheetsService
from .nlp_processor import NLPProcessor
from .clustering import FeedbackClusterer
from .ai_service import FeedbackSummarizer

__all__ = [
    'GoogleSheetsService',
    'NLPProcessor',
    'FeedbackClusterer',
    'FeedbackSummarizer'
]
```

ai_service.py

```
"""
AI Service - Hybrid Feedback Analysis Pipeline
Domain-agnostic: works for any type of customer/user feedback

API call budget per analysis run: MAX 3 calls
Call 1 → _summarize_clusters_batched (all clusters in one prompt)
Call 2 → _generate_final_summary
Call 3 → _generate_suggestions
"""

import re
import time
import google.generativeai as genai
from app.config import Config
from app.services.nlp_processor import NLPProcessor
from app.services.clustering import FeedbackClusterer
import logging
import numpy as np

logger = logging.getLogger(__name__)
```

Source-Code

— Retry configuration

```
_MAX_RETRIES = 0  # retry attempts after first failure
_BASE_BACKOFF = 5  # seconds — used when API doesn't return a retry delay

def _call_with_retry(model, prompt: str) -> str | None:
    """
    Call model.generate_content(prompt) with automatic retry on 429.
    Respects the retry_delay returned by the Gemini API when available.
    Returns the response text on success, or None on permanent failure.
    """
    for attempt in range(1 + _MAX_RETRIES):
        try:
            time.sleep(2) # brief pause before first attempt to avoid immediate rate limits
            response = model.generate_content(prompt)
            return response.text if response and response.text else None

        except Exception as e:
            err_str = str(e)

            # — 403: project access denied — retrying won't help —————
            if "403" in err_str:
                logger.error(f"Gemini 403 – project access denied. Check your API key / billing.")
                return None

            # — 429: rate limited — wait the suggested delay, then retry ———
            if "429" in err_str:
                if attempt == _MAX_RETRIES:
                    logger.error(f"Gemini 429 – quota exhausted after {_MAX_RETRIES} retries.")
                    return None

                # Parse the suggested delay from the error body if present
                match = re.search(r'retry_delay\s*\{\s*seconds:\s*(\d+)\}', err_str)
                wait = int(match.group(1)) + 2 if match else _BASE_BACKOFF * (2 ** attempt)

                logger.warning(
                    f"Gemini 429 – rate limited. Waiting {wait}s before retry "
                    f"({attempt}/{attempt + 1}/{_MAX_RETRIES})..."
                )
                time.sleep(wait)
```

Source-Code

```
        continue

# — Any other error —————
logger.error(f"Gemini API error: {e}")
return None

return None

class FeedbackSummarizer:
    def __init__(self):
        try:
            genai.configure(api_key=Config.GOOGLE_AI_API_KEY)
            self.model = genai.GenerativeModel(Config.AI_MODEL_NAME)

            self.nlp_processor = NLPProcessor()
            self.clusterer = FeedbackClusterer()

            logger.info("Hybrid AI Service initialized successfully")

        except Exception as e:
            logger.error(f"Failed to initialize Hybrid AI Service: {e}")
            raise

# — Utility —————

    def _convert_numpy_types(self, obj):
        """Convert numpy types to native Python types for JSON serialization."""
        if isinstance(obj, dict):
            return {self._convert_numpy_types(k): self._convert_numpy_types(v) for k, v in
obj.items()}
        elif isinstance(obj, list):
            return [self._convert_numpy_types(item) for item in obj]
        elif isinstance(obj, np.integer):
            return int(obj)
        elif isinstance(obj, np.floating):
            return float(obj)
        elif isinstance(obj, np.ndarray):
            return obj.tolist()
        return obj
```

Source-Code

— Main pipeline —

```
def get_structured_analysis(self, feedback_list):
    """
    HYBRID ANALYSIS PIPELINE
    Total Gemini API calls per run: 3 (regardless of cluster count).
    """
    if not feedback_list:
        return self._empty_response()

    try:
        logger.info(f"🚀 Starting hybrid analysis for {len(feedback_list)} feedbacks")

        # STEP 1 – Preprocess
        logger.info("Step 1: Preprocessing feedback...")
        cleaned_texts, embeddings =
self.nlp_processor.preprocess_feedback(feedback_list)

        # STEP 2 – Keyword extraction
        logger.info("Step 2: Extracting keywords...")
        keyword_results = self.nlp_processor.extract_keywords(feedback_list, top_n=3)
        keybert_themes = self.nlp_processor.aggregate_keywords(keyword_results)
        tfidf_themes = self.nlp_processor.extract_keywords_tfidf(feedback_list,
top_n=10)
        key_themes = self._merge_themes(keybert_themes, tfidf_themes)

        # STEP 3 – Clustering
        logger.info("Step 3: Clustering feedback...")
        cluster_result = self.clusterer.cluster_feedback(embeddings, feedback_list)

        # STEP 4 – Batch cluster summarisation ← API CALL 1
        logger.info(f"Step 4: Batch-summarising {cluster_result['n_clusters']} clusters (1
API call)...")
        cluster_summaries =
self._summarize_clusters_batched(cluster_result['clusters'])

        # STEP 5 – Final summary ← API CALL 2
        logger.info("Step 5: Generating final aggregated summary...")
        final_summary = self._generate_final_summary(
            cluster_summaries, key_themes, len(feedback_list)
        )
```

Source-Code

```
# STEP 6 – Suggestions ← API CALL 3
```

```
logger.info("Step 6: Generating detailed recommendations...")
```

```
suggestions = self._generate_suggestions(key_themes, cluster_summaries)
```

```
statistics = {  
    'total_responses': int(len(feedback_list)),  
    'clusters_found': int(cluster_result['n_clusters']),  
    'themes_identified': int(len(key_themes)),  
}
```

```
cluster_summaries_clean = {  
    str(cid): {  
        'summary': info['summary'],  
        'size': int(info['size']),  
        'sample_feedback': info['sample_feedback'],  
    }  
    for cid, info in cluster_summaries.items()  
}
```

```
result = {  
    'success': True,  
    'total_feedback': int(len(feedback_list)),  
    'summary': final_summary,  
    'key_themes': key_themes,  
    'statistics': statistics,  
    'suggestions': suggestions,  
    'cluster_info': {  
        'n_clusters': int(cluster_result['n_clusters']),  
        'cluster_summaries': cluster_summaries_clean,  
    },  
}
```

```
logger.info("✅ Hybrid analysis completed successfully")
```

```
return self._convert_numpy_types(result)
```

```
except Exception as e:
```

```
    logger.error(f"Error in hybrid analysis: {e}", exc_info=True)
```

```
    return self._empty_response()
```


Source-Code

— Theme merging

```
def _merge_themes(self, keybert_themes, tfidf_themes):
    merged = {}
    for theme, count in keybert_themes.items():
        merged[theme] = int(count)
    for theme, score in tfidf_themes.items():
        if theme not in merged:
            merged[theme] = int(score)
        else:
            merged[theme] = int((merged[theme] + score) // 2)
    return dict(sorted(merged.items(), key=lambda x: x[1], reverse=True)[:10])
```

— Cluster summarisation — BATCHED (1 API call) —————

```
def _summarize_clusters_batched(self, clusters: dict) -> dict:
    """
    Summarise ALL clusters in a single Gemini API call instead of one call
    per cluster. This is the primary fix for the 429 quota errors.
    Falls back to NLP-only heuristics when the API is unavailable.
    """
    if not clusters:
        return {}

    cluster_blocks = []
    for cluster_id, feedbacks in clusters.items():
        # Cap at 5 samples per cluster — keeps total prompt well under TPM limits
        # (8 clusters × 5 samples × ~20 words ≈ 800 tokens vs 15 samples ≈ 2400 tokens)
        samples = "\n".join(f" - {fb[:150]}" for fb in feedbacks[:5])
        cluster_blocks.append(
            f"#### CLUSTER {cluster_id} ({len(feedbacks)} feedbacks)\n{samples}"
        )

    combined = "\n\n".join(cluster_blocks)

    prompt = f"""
    You are analysing customer feedback that has been grouped into semantic clusters.
    For EACH cluster below, provide a 2-sentence summary covering:
    1. The main theme or topic of this group
    2. The overall tone (positive / negative / neutral / mixed)
```

Source-Code

Respond in this EXACT format for every cluster — no extra text:

CLUSTER <id>: <2-sentence summary>

{combined}

"""

```
raw = _call_with_retry(self.model, prompt)
```

```
if raw:
```

```
    return self._parse_batched_summaries(raw, clusters)
```

```
    logger.warning("Batched cluster summarisation failed — using NLP fallback  
summaries.")
```

```
    return self._fallback_cluster_summaries(clusters)
```

```
def _parse_batched_summaries(self, raw: str, clusters: dict) -> dict:
```

```
    """Parse the single batched API response into a per-cluster dict."""
```

```
    pattern = re.compile(
```

```
        r'CLUSTER\s+(\d+)\s*:\s*(.+?)(?=\nCLUSTER\s+\d+\s*:|$)',
```

```
        re.DOTALL
```

```
)
```

```
    parsed = {
```

```
        int(cid): text.strip().replace('\n', ' ')
```

```
        for cid, text in pattern.findall(raw)
```

```
    }
```

```
    summaries = {}
```

```
    for cluster_id, feedbacks in clusters.items():
```

```
        cid_int = int(cluster_id)
```

```
        summaries[cid_int] = {
```

```
            'summary':    parsed.get(cid_int, f"Group of {len(feedbacks)} related feedback  
items."),
```

```
            'size':       len(feedbacks),
```

```
            'sample_feedback': feedbacks[0] if feedbacks else "",
```

```
        }
```

```
    logger.info(f"✅ Parsed batched summaries for {len(summaries)} clusters")
```

Source-Code

```
return summaries

def _fallback_cluster_summaries(self, clusters: dict) -> dict:
    """Pure-NLP fallback — zero API calls."""
    summaries = {}
    neg_words = ['bad', 'poor', 'slow', 'broken', 'issue', 'problem', 'fail', 'disappoint']
    pos_words = ['good', 'great', 'love', 'excellent', 'fast', 'easy', 'helpful']

    for cluster_id, feedbacks in clusters.items():
        cid_int = int(cluster_id)
        sample_text = ' '.join(feedbacks[:3]).lower()
        tone = (
            'negative' if any(w in sample_text for w in neg_words) else
            'positive' if any(w in sample_text for w in pos_words) else
            'mixed'
        )
        summaries[cid_int] = {
            'summary': f"Group of {len(feedbacks)} feedback items with a {tone} overall",
            'size': len(feedbacks),
            'sample_feedback': feedbacks[0] if feedbacks else "",
        }
    return summaries

# — Final summary (1 API call) —————

def _generate_final_summary(self, cluster_summaries, themes, total_count):
    cluster_texts = [
        f"Group {cid + 1} ({info['size']} feedbacks): {info['summary']}"
        for cid, info in cluster_summaries.items()
    ]
    combined_clusters = "\n".join(cluster_texts)
    top_themes_str = ", ".join(list(themes.keys())[:5])

    prompt = f"""
Based on analysing {total_count} customer feedbacks grouped into
{len(cluster_summaries)} semantic clusters:

Cluster insights:
{combined_clusters}
```

Source-Code

Top themes: {top_themes_str}

Provide a CONCISE executive summary in exactly 7-8 lines (NOT MORE).

Focus on:

- Overall sentiment/trend (1-2 lines)
- Top 2-3 key findings (3-4 lines)
- Primary recommendation (1-2 lines)

Write in paragraph format, NOT bullet points. Keep it brief, professional, and actionable.

"""

```
result = _call_with_retry(self.model, prompt)
```

```
if result:
```

```
    logger.info("✅ Final summary generated")
```

```
    return result
```

```
logger.warning("Final summary API call failed — using fallback.")
```

```
return self._generate_fallback_summary(cluster_summaries, themes, total_count)
```

```
def _generate_fallback_summary(self, cluster_summaries, themes, total_count):
```

```
    top_themes = ", ".join(list(themes.keys())[:3])
```

```
    summaries_text = " ".join([info['summary'].lower() for info in  
cluster_summaries.values()])
```

```
    if 'positive' in summaries_text or 'praise' in summaries_text or 'excellent' in  
summaries_text:
```

```
        tone = "generally positive with some areas for improvement"
```

```
    elif 'negative' in summaries_text or 'complaint' in summaries_text or 'poor' in  
summaries_text:
```

```
        tone = "mixed with significant concerns noted"
```

```
    else:
```

```
        tone = "varied across different aspects"
```

```
    summary = f"Analysed {total_count} customer feedbacks across  
{len(cluster_summaries)} thematic groups. "
```

```
    summary += f"The overall feedback is {tone}. "
```

```
    summary += f"Key themes include {top_themes}, which appear most frequently  
across responses. "
```

```
    if len(cluster_summaries) > 5:
```

```
        summary += "The diverse range of topics indicates customers focus on multiple  
aspects of the experience. "
```

Source-Code

```
else:
    summary += "Feedback is concentrated around a few specific areas requiring
attention. "
    top_theme = list(themes.keys())[0] if themes else "identified issues"
    summary += f"Primary recommendation: Address the most frequently mentioned
theme ({top_theme.lower()}) "
    summary += "to improve customer satisfaction. Regular monitoring of feedback
trends is advised."
    return summary
```

— Suggestions (1 API call) —————

```
def _generate_suggestions(self, themes, cluster_summaries):
    cluster_insights = [
        f"Cluster {cid + 1}: {info['summary']}"
        for cid, info in cluster_summaries.items()
    ]
    combined_insights = "\n".join(cluster_insights)
    top_themes = ", ".join(list(themes.keys())[:5])
```

```
prompt = f"""
```

Based on customer feedback analysis with these insights:

{combined_insights}

Top themes mentioned: {top_themes}

Generate 4-5 SPECIFIC, ACTIONABLE recommendations directly derived from the feedback above.

Each recommendation must:

1. Start with a relevant emoji icon
2. Have a clear title describing what to improve
3. Include 2 lines — SPECIFIC actions based strictly on the themes in the feedback, not generic advice

Format EXACTLY as:

[Emoji] [Title]

[Line 1: Specific action to take]

[Line 2: Why it matters or additional detail]

Each recommendation should be separated by a blank line.

Source-Code

"""

```
result = _call_with_retry(self.model, prompt)
if result:
    suggestions = self._parse_recommendations(result)
    logger.info(f"✅ Generated {len(suggestions)} recommendations")
    return suggestions[:5]

logger.warning("Suggestions API call failed — using fallback.")
return self._generate_fallback_suggestions(themes, cluster_summaries)
```

```
def _parse_recommendations(self, ai_response):
    suggestions = []
    current_recommendation = []

    for line in ai_response.strip().split('\n'):
        line = line.strip()
        if not line:
            if current_recommendation:
                rec_text = '\n'.join(current_recommendation)
                if len(rec_text) > 10:
                    suggestions.append(rec_text)
                    current_recommendation = []
            else:
                current_recommendation.append(line.lstrip('0123456789. '))

        if current_recommendation:
            rec_text = '\n'.join(current_recommendation)
            if len(rec_text) > 10:
                suggestions.append(rec_text)

    return suggestions
```

```
def _generate_fallback_suggestions(self, themes, cluster_summaries):
    """Domain-agnostic fallback — no hardcoded domain keywords."""
    suggestions = []
    all_summaries = " ".join([info['summary'].lower() for info in
cluster_summaries.values()])
    top_theme_list = list(themes.keys())[:5]

    neg_signals = ['poor', 'bad', 'slow', 'broken', 'confusing', 'difficult',
```

Source-Code

```
'frustrat', 'disappoint', 'complaint', 'issue', 'problem',
'fail', 'error', 'bug', 'crash', 'rude', 'unhelpful']
pos_signals = ['great', 'love', 'excellent', 'amazing', 'fast', 'easy',
'helpful', 'smooth', 'intuitive', 'reliable', 'good']

neg_count = sum(1 for s in neg_signals if s in all_summaries)
pos_count = sum(1 for s in pos_signals if s in all_summaries)

if top_theme_list:
    primary = top_theme_list[0]
    suggestions.append(
        f"🎯 Prioritise Improvements Around \"{primary.title()}\n"
        f"Review all feedback mentioning \"{primary.lower()}\" and identify root causes "
        f"behind recurring concerns in this area.\n"
        f"Create a dedicated action plan with measurable targets and assign clear
ownership."
    )

    if len(top_theme_list) >= 2:
        secondary = top_theme_list[1]
        suggestions.append(
            f"🔍 Deep-Dive Into \"{secondary.title()}\" Feedback\n"
            f"Segment all responses related to \"{secondary.lower()}\" to uncover specific
pain points.\n"
            f"Use these findings to inform the next product/service iteration or training
cycle."
        )

    if neg_count > pos_count:
        suggestions.append(
            f"⚠️ Address Recurring Negative giExperiences\n"
            f"Identify the top 3 most-cited negative issues from the clusters and define clear
"
            f"resolution steps for each within the next review cycle.\n"
            f"Set up an escalation path so critical complaints are triaged and resolved
faster."
        )
    elif pos_count > 0:
        suggestions.append(
            f"★ Amplify What Is Already Working Well\n"
```

Source-Code

```
"Identify aspects receiving the most positive mentions and document what
makes them "
    "successful so they can be replicated elsewhere.\n"
    "Highlight these wins internally to motivate teams and externally to build user
trust."
)

if len(cluster_summaries) > 2:
    suggestions.append(
        "🔄 Standardise Processes to Reduce Variability\n"
        f"Feedback spans {len(cluster_summaries)} distinct themes, suggesting
inconsistency in delivery.\n"
        "Document best-practice workflows and conduct regular audits to ensure a
consistent experience."
    )

    suggestions.append(
        "🇮🇹 Implement Continuous Feedback Monitoring\n"
        "Set up weekly feedback review sessions to identify emerging issues before they
escalate.\n"
        "Use trend dashboards to proactively address concerns before they become
widespread problems."
    )

    return suggestions[:5]

# — Empty response
```

```
def _empty_response(self):
    return {
        'success': False,
        'total_feedback': 0,
        'summary': 'No feedback data to analyse',
        'key_themes': {},
        'statistics': {
            'total_responses': 0,
            'clusters_found': 0,
            'themes_identified': 0,
        },
        'suggestions': ['No suggestions available'],
    }
```


Source-Code

```
'cluster_info': {
    'n_clusters': 0,
    'cluster_summaries': {},
},
}
```

clustering.py

```
"""
```

```
Clustering Module for Feedback Grouping
Uses KMeans for semantic clustering
```

```
"""
```

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import numpy as np
import logging
```

```
logger = logging.getLogger(__name__)
```

```
class FeedbackClusterer:
```

```
    def __init__(self):
        """Initialize clustering algorithms"""
        pass
```

```
    def cluster_feedback(self, embeddings, feedback_list, n_clusters=None):
```

```
        """
```

```
        Cluster feedback based on semantic embeddings using KMeans
```

```
        """
```

```
        # Validate inputs
```

```
        if len(embeddings) == 0:
```

```
            logger.warning("⚠️ No embeddings provided for clustering")
```

```
            return {'clusters': {}, 'labels': [], 'n_clusters': 0}
```

```
        # Check if embeddings is a valid numpy array
```

```
        if not isinstance(embeddings, np.ndarray):
```

```
            logger.error(f"⚠️ Embeddings is not a numpy array: {type(embeddings)}")
```

```
            return {'clusters': {}, 'labels': [], 'n_clusters': 0}
```

```
        # Check embeddings shape
```

Source-Code

```
if len(embeddings.shape) != 2:
    logger.error(f" ⚠️ Invalid embeddings shape: {embeddings.shape}")
    return {'clusters': {}, 'labels': [], 'n_clusters': 0}

if embeddings.shape[0] != len(feedback_list):
    logger.error(f" ⚠️ Embeddings count ({embeddings.shape[0]}) doesn't match
feedback count ({len(feedback_list)})")
    return {'clusters': {}, 'labels': [], 'n_clusters': 0}

try:
    # Auto-detect optimal number of clusters
    if n_clusters is None:
        n_clusters = self._optimal_clusters(embeddings)
        logger.info(f" 🇮🇹 Optimal clusters detected: {n_clusters}")

    # Ensure at least 2 clusters, max 10, but also ensure we have enough data
    # For large datasets, use a percentage-based approach
    max_clusters = min(10, max(2, len(feedback_list) // 3))
    n_clusters = max(2, min(n_clusters, max_clusters))

    logger.info(f" 🇮🇹 Using {n_clusters} clusters for {len(feedback_list)} feedback
items")

    # Perform KMeans clustering
    labels = self._kmeans_clustering(embeddings, n_clusters)

    if labels is None or len(labels) == 0:
        logger.error(f" ⚠️ KMeans returned empty labels")
        return {'clusters': {}, 'labels': [], 'n_clusters': 0}

    # Organize feedback by clusters
    clusters = self._organize_clusters(feedback_list, labels)

    # Filter out empty clusters
    clusters = {k: v for k, v in clusters.items() if len(v) > 0}

    logger.info(f" ✅ Clustered {len(feedback_list)} feedbacks into {len(clusters)}
clusters")

    if len(clusters) == 0:
```

Source-Code

```
logger.warning(" ⚠️ No clusters formed after organization")
return {'clusters': {}, 'labels': labels.tolist() if hasattr(labels, 'tolist') else labels,
'n_clusters': 0}

return {
    'clusters': clusters,
    'labels': labels.tolist() if hasattr(labels, 'tolist') else labels,
    'n_clusters': len(clusters)
}

except Exception as e:
    logger.error(f" ❌ Error in clustering: {e}", exc_info=True)
    # Fallback: single cluster
    return {
        'clusters': {0: feedback_list},
        'labels': [0] * len(feedback_list),
        'n_clusters': 1
    }

def _optimal_clusters(self, embeddings):
    """Automatically determine optimal number of clusters"""
    try:
        n_samples = len(embeddings)

        if n_samples < 2:
            logger.warning(f" ⚠️ Too few samples ({n_samples}) for clustering")
            return 1

        # Rule of thumb: sqrt(n/2) for small datasets, log-based for large datasets
        if n_samples <= 100:
            optimal_k = int(np.sqrt(n_samples / 2))
            optimal_k = max(2, min(optimal_k, 8))

            # Try silhouette method for small datasets
            best_k = optimal_k
            best_score = -1

            for k in range(2, min(8, n_samples // 2)):
                try:
                    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
```

Source-Code

```
labels = kmeans.fit_predict(embeddings)
score = silhouette_score(embeddings, labels)

if score > best_score:
    best_score = score
    best_k = k
except Exception as e:
    logger.debug(f"Silhouette score calculation failed for k={k}: {e}")
    continue

logger.info(f"🇮🇹 Optimal clusters (silhouette): {best_k} (score:
{best_score:.3f})")
return best_k
else:
    # For large datasets, use logarithmic scaling
    # This gives: 100->4, 500->6, 1000->7, 5000->9
    optimal_k = int(np.log2(n_samples) + 2)
    optimal_k = max(3, min(optimal_k, 10))
    logger.info(f"🇮🇹 Optimal clusters (log-based): {optimal_k} for {n_samples}
samples")
    return optimal_k

except Exception as e:
    logger.warning(f"Error in optimal cluster detection: {e}", exc_info=True)
    return 3 # Safe default

def _kmeans_clustering(self, embeddings, n_clusters):
    """Perform KMeans clustering"""
    try:
        # Ensure embeddings is 2D
        if len(embeddings.shape) == 1:
            embeddings = embeddings.reshape(-1, 1)

        # Check for NaN or Inf values
        if np.any(np.isnan(embeddings)) or np.any(np.isinf(embeddings)):
            logger.error("⚠️ Embeddings contain NaN or Inf values")
            return np.zeros(len(embeddings), dtype=int)

        # Ensure we don't have more clusters than samples
        if n_clusters > len(embeddings):
```

Source-Code

```
        logger.warning(f" ⚠ Reducing clusters from {n_clusters} to {len(embeddings)}  
(number of samples)")  
        n_clusters = len(embeddings)  
  
        kmeans = KMeans(  
            n_clusters=n_clusters,  
            random_state=42,  
            n_init=10,  
            max_iter=300  
        )  
        labels = kmeans.fit_predict(embeddings)  
  
        unique_labels = len(np.unique(labels))  
        logger.info(f" ✅ KMeans clustering completed: {unique_labels} unique clusters  
found")  
  
        return labels  
  
    except Exception as e:  
        logger.error(f" ❌ KMeans clustering error: {e}", exc_info=True)  
        return np.zeros(len(embeddings), dtype=int)  
  
def _organize_clusters(self, feedback_list, labels):  
    """Organize feedback into cluster groups"""  
    clusters = {}  
  
    for idx, label in enumerate(labels):  
        if label not in clusters:  
            clusters[label] = []  
        clusters[label].append(feedback_list[idx])  
  
    return clusters
```

export_service.py

```
"""
```

```
Export Service for generating PDF and Excel reports
```

```
"""
```

```
import io
```

```
from datetime import datetime
```

Source-Code

```
from reportlab.lib import colors
from reportlab.lib.pagesizes import letter, A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table,
TableStyle, PageBreak, Image
from reportlab.pdfgen import canvas
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from openpyxl.chart import BarChart, Reference
import logging
import base64
```

```
logger = logging.getLogger(__name__)
```

```
class ExportService:
```

```
    """Service for exporting analysis reports"""
```

```
    @staticmethod
```

```
    def generate_pdf_report(analysis_data, metadata):
```

```
        """
```

```
        Generate PDF report from analysis data
```

```
        Returns: BytesIO object with PDF content
```

```
        """
```

```
        buffer = io.BytesIO()
```

```
        doc = SimpleDocTemplate(buffer, pagesize=letter, topMargin=0.5*inch)
```

```
        story = []
```

```
        styles = getSampleStyleSheet()
```

```
        # Custom styles
```

```
        title_style = ParagraphStyle(
```

```
            'CustomTitle',
```

```
            parent=styles['Heading1'],
```

```
            fontSize=24,
```

```
            textColor=colors.HexColor('#667eea'),
```

```
            spaceAfter=30,
```

```
            alignment=1 # Center
```

```
        )
```

```
        heading_style = ParagraphStyle(
```

```
            'CustomHeading',
```

Source-Code

```
parent=styles['Heading2'],
fontSize=16,
textColor=colors.HexColor('#667eea'),
spaceAfter=12,
spaceBefore=12
)

# Title
story.append(Paragraph("📊 Feedback Analysis Report", title_style))
story.append(Spacer(1, 0.2*inch))

# Metadata Section
story.append(Paragraph("Report Information", heading_style))
metadata_text = f"""
<b>Generated:</b> {datetime.fromisoformat(analysis_data.get('timestamp',
datetime.utcnow().isoformat()))}.strftime('%Y-%m-%d %H:%M:%S')}<br/>
<b>Spreadsheet ID:</b> {metadata.get('spreadsheet_id', 'N/A')}<br/>
<b>Range Analyzed:</b> {metadata.get('range_analyzed', 'N/A')}<br/>
<b>AI Model:</b> {metadata.get('ai_model', 'N/A')}<br/>
<b>Analysis Type:</b> {metadata.get('analysis_type', 'N/A')}<br/>
<b>Version:</b> {metadata.get('version', 'N/A')}
"""
story.append(Paragraph(metadata_text, styles['Normal']))
story.append(Spacer(1, 0.3*inch))

# Statistics Section
story.append(Paragraph("📈 Statistics", heading_style))
stats = analysis_data.get('statistics', {})
stats_data = [
    ['Metric', 'Value'],
    ['Total Feedback', str(analysis_data.get('total_feedback', 0))],
    ['Clusters Found', str(stats.get('clusters_found', 0))],
    ['Themes Identified', str(stats.get('themes_identified', 0))]
]
stats_table = Table(stats_data, colWidths=[3*inch, 2*inch])
stats_table.setStyle(TableStyle([
    ('BACKGROUND', (0, 0), (-1, 0), colors.HexColor('#667eea')),
    ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
    ('ALIGN', (0, 0), (-1, -1), 'LEFT'),
    ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
    ('FONTSIZE', (0, 0), (-1, 0), 12),
```

Source-Code

```
('BOTTOMPADDING', (0, 0), (-1, 0), 12),
('BACKGROUND', (0, 1), (-1, -1), colors.beige),
('GRID', (0, 0), (-1, -1), 1, colors.grey),
('ROWBACKGROUNDS', (0, 1), (-1, -1), [colors.white, colors.lightgrey])
]))
story.append(stats_table)
story.append(Spacer(1, 0.3*inch))

# Summary Section
story.append(Paragraph("📄 AI Summary", heading_style))
summary = analysis_data.get('summary', 'No summary available')
# Split summary into paragraphs
for para in summary.split('\n'):
    if para.strip():
        story.append(Paragraph(para.strip(), styles['Normal']))
        story.append(Spacer(1, 0.1*inch))
story.append(Spacer(1, 0.2*inch))

# Key Themes Section
story.append(Paragraph("🎯 Key Themes", heading_style))
themes = analysis_data.get('key_themes', {})
if themes:
    themes_data = [['Theme', 'Mentions']]
    for theme, count in sorted(themes.items(), key=lambda x: x[1], reverse=True):
        themes_data.append([theme.title(), str(count)])

themes_table = Table(themes_data, colWidths=[4*inch, 1*inch])
themes_table.setStyle(TableStyle([
    ('BACKGROUND', (0, 0), (-1, 0), colors.HexColor('#667eea')),
    ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
    ('ALIGN', (0, 0), (-1, -1), 'LEFT'),
    ('ALIGN', (1, 0), (1, -1), 'CENTER'),
    ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
    ('FONTSIZE', (0, 0), (-1, 0), 12),
    ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
    ('BACKGROUND', (0, 1), (-1, -1), colors.beige),
    ('GRID', (0, 0), (-1, -1), 1, colors.grey),
]))
story.append(themes_table)
story.append(Spacer(1, 0.3*inch))
```


Source-Code

```
# Recommendations Section
story.append(Paragraph("💡 Recommendations", heading_style))
suggestions = analysis_data.get('suggestions', [])
for idx, suggestion in enumerate(suggestions, 1):
    lines = suggestion.split('\n')
    title = lines[0] if lines else suggestion
    description = '\n'.join(lines[1:]) if len(lines) > 1 else ""

    story.append(Paragraph(f"<b>{idx}. {title}</b>", styles['Normal']))
    if description:
        story.append(Paragraph(description, styles['Normal']))
        story.append(Spacer(1, 0.15*inch))
story.append(Spacer(1, 0.2*inch))

# Cluster Details Section
cluster_info = analysis_data.get('cluster_info', {})
if cluster_info.get('cluster_summaries'):
    story.append(Paragraph("🔍 Cluster Analysis", heading_style))
    clusters = cluster_info.get('cluster_summaries', {})
    for cluster_id, cluster in clusters.items():
        cluster_title = f"Cluster {int(cluster_id) + 1} ({cluster.get('size', 0)} feedbacks)"
        story.append(Paragraph(f"<b>{cluster_title}</b>", styles['Normal']))
        story.append(Paragraph(cluster.get('summary', 'No summary'),
styles['Normal']))
        if cluster.get('sample_feedback'):
            story.append(Paragraph(f"<i>Sample: {cluster.get('sample_feedback',
'')[[:200]]}...</i>", styles['Normal']))
            story.append(Spacer(1, 0.2*inch))

# Build PDF
doc.build(story)
buffer.seek(0)
return buffer

@staticmethod
def generate_excel_report(analysis_data, metadata):
    """
    Generate Excel report from analysis data
    Returns: BytesIO object with Excel content
    """
    wb = Workbook()
```

Source-Code

```
ws = wb.active
ws.title = "Analysis Report"

# Styling
header_fill = PatternFill(start_color="667eea", end_color="667eea", fill_type="solid")
header_font = Font(bold=True, color="FFFFFF", size=12)
title_font = Font(bold=True, size=16, color="667eea")
border = Border(
    left=Side(style='thin'),
    right=Side(style='thin'),
    top=Side(style='thin'),
    bottom=Side(style='thin')
)
center_align = Alignment(horizontal='center', vertical='center')

row = 1

# Title
ws.merge_cells(f'A{row}:B{row}')
ws[f'A{row}'] = " 🇮🇹 Feedback Analysis Report"
ws[f'A{row}'].font = title_font
ws[f'A{row}'].alignment = center_align
row += 2

# Metadata
ws[f'A{row}'] = "Report Information"
ws[f'A{row}'].font = header_font
ws[f'A{row}'].fill = header_fill
ws.merge_cells(f'A{row}:B{row}')
row += 1

gen_time = datetime.fromisoformat(analysis_data.get('timestamp',
datetime.utcnow().isoformat())).strftime('%Y-%m-%d %H:%M:%S')
metadata_rows = [
    ['Generated', gen_time],
    ['Spreadsheet ID', metadata.get('spreadsheet_id', 'N/A')],
    ['Range Analyzed', metadata.get('range_analyzed', 'N/A')],
    ['AI Model', metadata.get('ai_model', 'N/A')],
    ['Analysis Type', metadata.get('analysis_type', 'N/A')],
    ['Version', metadata.get('version', 'N/A')]
]
```

Source-Code

```
for key, value in metadata_rows:
    ws[f'A{row}'] = key
    ws[f'B{row}'] = value
    ws[f'A{row}'].font = Font(bold=True)
    ws[f'A{row}'].border = border
    ws[f'B{row}'].border = border
    row += 1

row += 1

# Statistics
ws[f'A{row}'] = "Statistics"
ws[f'A{row}'].font = header_font
ws[f'A{row}'].fill = header_fill
ws.merge_cells(f'A{row}:B{row}')
row += 1

stats = analysis_data.get('statistics', {})
stats_rows = [
    ['Metric', 'Value'],
    ['Total Feedback', analysis_data.get('total_feedback', 0)],
    ['Clusters Found', stats.get('clusters_found', 0)],
    ['Themes Identified', stats.get('themes_identified', 0)]
]

for stat_row in stats_rows:
    ws[f'A{row}'] = stat_row[0]
    ws[f'B{row}'] = stat_row[1]
    ws[f'A{row}'].font = Font(bold=True) if row == 0 else Font()
    ws[f'A{row}'].border = border
    ws[f'B{row}'].border = border
    row += 1

row += 1

# Summary
ws[f'A{row}'] = "AI Summary"
ws[f'A{row}'].font = header_font
ws[f'A{row}'].fill = header_fill
ws.merge_cells(f'A{row}:B{row}')
```

Source-Code

```
row += 1

summary = analysis_data.get('summary', 'No summary available')
ws[f'A{row}'] = summary
ws.merge_cells(f'A{row}:B{row}')
ws[f'A{row}'].alignment = Alignment(wrap_text=True, vertical='top')
ws[f'A{row}'].border = border
# Auto-adjust row height
ws.row_dimensions[row].height = len(summary.split('\n')) * 15
row += 2

# Key Themes
themes = analysis_data.get('key_themes', {})
if themes:
    ws[f'A{row}'] = "Key Themes"
    ws[f'A{row}'].font = header_font
    ws[f'A{row}'].fill = header_fill
    ws.merge_cells(f'A{row}:B{row}')
    row += 1

    ws[f'A{row}'] = "Theme"
    ws[f'B{row}'] = "Mentions"
    ws[f'A{row}'].font = Font(bold=True)
    ws[f'B{row}'].font = Font(bold=True)
    ws[f'A{row}'].fill = PatternFill(start_color="E0E0E0", end_color="E0E0E0",
fill_type="solid")
    ws[f'B{row}'].fill = PatternFill(start_color="E0E0E0", end_color="E0E0E0",
fill_type="solid")
    ws[f'A{row}'].border = border
    ws[f'B{row}'].border = border
    row += 1

for theme, count in sorted(themes.items(), key=lambda x: x[1], reverse=True):
    ws[f'A{row}'] = theme.title()
    ws[f'B{row}'] = count
    ws[f'A{row}'].border = border
    ws[f'B{row}'].border = border
    row += 1

row += 1
```

Source-Code

```
# Recommendations
suggestions = analysis_data.get('suggestions', [])
if suggestions:
    ws[f'A{row}'] = "Recommendations"
    ws[f'A{row}'].font = header_font
    ws[f'A{row}'].fill = header_fill
    ws.merge_cells(f'A{row}:B{row}')
    row += 1

for idx, suggestion in enumerate(suggestions, 1):
    lines = suggestion.split('\n')
    title = lines[0] if lines else suggestion
    description = '\n'.join(lines[1:]) if len(lines) > 1 else ""

    ws[f'A{row}'] = f"{idx}. {title}"
    ws[f'A{row}'].font = Font(bold=True)
    ws.merge_cells(f'A{row}:B{row}')
    ws[f'A{row}'].border = border
    row += 1

    if description:
        ws[f'A{row}'] = description
        ws.merge_cells(f'A{row}:B{row}')
        ws[f'A{row}'].alignment = Alignment(wrap_text=True, vertical='top')
        ws[f'A{row}'].border = border
        ws.row_dimensions[row].height = len(description.split('\n')) * 15
        row += 1

    row += 1

# Cluster Details
cluster_info = analysis_data.get('cluster_info', {})
if cluster_info.get('cluster_summaries'):
    ws[f'A{row}'] = "Cluster Analysis"
    ws[f'A{row}'].font = header_font
    ws[f'A{row}'].fill = header_fill
    ws.merge_cells(f'A{row}:B{row}')
    row += 1

clusters = cluster_info.get('cluster_summaries', {})
for cluster_id, cluster in clusters.items():
```

Source-Code

```
cluster_title = f"Cluster {int(cluster_id) + 1} ({cluster.get('size', 0)} feedbacks)"
ws[f'A{row}'] = cluster_title
ws[f'A{row}'].font = Font(bold=True)
ws.merge_cells(f'A{row}:B{row}')
ws[f'A{row}'].border = border
row += 1

ws[f'A{row}'] = cluster.get('summary', 'No summary')
ws.merge_cells(f'A{row}:B{row}')
ws[f'A{row}'].alignment = Alignment(wrap_text=True, vertical='top')
ws[f'A{row}'].border = border
ws.row_dimensions[row].height = len(cluster.get('summary', '').split('\n')) * 15
row += 1

if cluster.get('sample_feedback'):
    ws[f'A{row}'] = f"Sample: {cluster.get('sample_feedback', '')[:200]}"
    ws.merge_cells(f'A{row}:B{row}')
    ws[f'A{row}'].font = Font(italic=True)
    ws[f'A{row}'].border = border
    row += 1

row += 1

# Adjust column widths
ws.column_dimensions['A'].width = 50
ws.column_dimensions['B'].width = 30

# Save to buffer
buffer = io.BytesIO()
wb.save(buffer)
buffer.seek(0)
return buffer
```

nlp_processor.py

```
"""
```

NLP Preprocessing Module

Handles text cleaning, embeddings, and keyword extraction

```
"""
```

```
import re
```

Source-Code

```
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sentence_transformers import SentenceTransformer
from keybert import KeyBERT
from sklearn.feature_extraction.text import TfidfVectorizer
from collections import Counter
import numpy as np
import logging

logger = logging.getLogger(__name__)

class NLPProcessor:
    def __init__(self):
        """Initialize NLP tools and models"""
        try:
            # Initialize NLTK tools
            self.stop_words = set(stopwords.words('english'))
            self.lemmatizer = WordNetLemmatizer()

            # Load embedding model (lightweight and fast)
            logger.info("Loading sentence embedding model...")
            self.embedding_model = SentenceTransformer('all-MiniLM-L6-v2')

            # Initialize KeyBERT for keyword extraction
            self.kw_model = KeyBERT(model=self.embedding_model)

            logger.info("✅ NLP Processor initialized successfully")

        except Exception as e:
            logger.error(f"Failed to initialize NLP Processor: {e}")
            raise

    def preprocess_feedback(self, feedback_list):
        """
        Clean and preprocess feedback texts
        Returns: cleaned texts and their embeddings
        """
        if not feedback_list:
            return [], np.array([])
```

Source-Code

```
try:
    cleaned_texts = []

    for raw in feedback_list:
        # 1. Lowercase
        text = (raw or '').lower()

        # 2. Remove special characters & numbers
        text = re.sub(r'[^\w\s]', '', text)

        # 3. Tokenize
        words = text.split()

        # 4. Remove stopwords
        words = [w for w in words if w not in self.stop_words and len(w) > 2]

        # 5. Lemmatize
        words = [self.lemmatizer.lemmatize(w) for w in words]

        # 6. Join back — never pass empty string to the encoder (hurts clustering)
        cleaned_text = ' '.join(words)
        if cleaned_text:
            cleaned_texts.append(cleaned_text)
        else:
            cleaned_texts.append((raw or '').strip() or '')

    # Generate embeddings for all cleaned feedback
    logger.info(f"Generating embeddings for {len(cleaned_texts)} texts...")
    embeddings = self.embedding_model.encode(cleaned_texts,
show_progress_bar=False)

    logger.info(f"✅ Preprocessing complete: {len(cleaned_texts)} texts processed")
    return cleaned_texts, embeddings

except Exception as e:
    logger.error(f"Error in preprocessing: {e}")
    return [], np.array([])

def extract_keywords(self, feedback_list, top_n=5):
    """
    Extract keywords using KeyBERT
```


Source-Code

```
"""
try:
    results = []

    for feedback in feedback_list:
        if not feedback or len(feedback.strip()) < 5:
            results.append({
                'feedback': feedback,
                'keywords': []
            })
            continue

        # Extract keywords with scores
        keywords = self.kw_model.extract_keywords(
            feedback,
            keyphrase_ngram_range=(1, 2),
            stop_words='english',
            top_n=top_n
        )

        # Extract only keyword strings
        keywords_only = [kw for kw, score in keywords]

        results.append({
            'feedback': feedback,
            'keywords': keywords_only
        })

    logger.info(f"✅ Keywords extracted from {len(results)} feedbacks")
    return results

except Exception as e:
    logger.error(f"Error extracting keywords: {e}")
    return []

def aggregate_keywords(self, keyword_results):
    """
    Aggregate and count keyword frequency across all feedback
    """
    try:
        all_keywords = []
```

Source-Code

```
for item in keyword_results:
    all_keywords.extend(item.get('keywords', []))

# Count frequency
keyword_freq = Counter(all_keywords)

# Return top keywords as dict
top_keywords = dict(keyword_freq.most_common(15))

logger.info(f"✅ Aggregated {len(top_keywords)} unique keywords")
return top_keywords

except Exception as e:
    logger.error(f"Error aggregating keywords: {e}")
    return {}

def extract_keywords_tfidf(self, feedback_list, top_n=10):
    """
    Alternative: Extract keywords using TF-IDF
    """
    try:
        if not feedback_list or len(feedback_list) == 0:
            return {}

        # Create TF-IDF vectorizer
        vectorizer = TfidfVectorizer(
            max_features=30,
            ngram_range=(1, 2),
            stop_words='english',
            min_df=1
        )

        # Fit and transform
        tfidf_matrix = vectorizer.fit_transform(feedback_list)
        feature_names = vectorizer.get_feature_names_out()

        # Get average TF-IDF scores
        avg_scores = np.mean(tfidf_matrix.toarray(), axis=0)

        # Create keyword-score pairs
```

Source-Code

```
keyword_scores = list(zip(feature_names, avg_scores))
keyword_scores.sort(key=lambda x: x[1], reverse=True)

# Return top keywords as dict
top_keywords = {
    kw.title(): int(score * 100)
    for kw, score in keyword_scores[:top_n]
    if score > 0
}

logger.info(f"✅ TF-IDF extracted {len(top_keywords)} keywords")
return top_keywords

except Exception as e:
    logger.error(f"Error in TF-IDF extraction: {e}")
    return {}
```

sheets_service.py

```
"""
Google Sheets Service - Using API Key authentication
"""

import pandas as pd
import requests
import logging
from app.config import Config

logger = logging.getLogger(__name__)

class GoogleSheetsService:
    def __init__(self):
        self.api_key = Config.GOOGLE_API_KEY
        self.base_url = "https://sheets.googleapis.com/v4/spreadsheets"

    def get_feedback_data(self, spreadsheet_id, range_name='Sheet1!A:B'):
        """
        Fetch feedback data from Google Sheets using API key
        """
        try:
            # Construct the API URL
```

Source-Code

```
url = f"{self.base_url}/{spreadsheet_id}/values/{range_name}"
params = {
    'key': self.api_key,
    'majorDimension': 'ROWS'
}

logger.info(f"Fetching data from sheet: {spreadsheet_id}, range: {range_name}")

# Make the API request
response = requests.get(url, params=params)
response.raise_for_status()

data = response.json()
values = data.get('values', [])

if not values:
    logger.warning("No data found in the specified range")
    return []

# Process the data
feedback_list = []

# Assuming first row contains headers
if len(values) > 1:
    headers = values[0]

    # Find feedback column (case insensitive)
    feedback_col_index = None
    for i, header in enumerate(headers):
        if 'feedback' in header.lower():
            feedback_col_index = i
            break

    if feedback_col_index is None:
        # If no 'feedback' column found, use the last column
        feedback_col_index = len(headers) - 1
        logger.info(f"No 'feedback' column found, using column index: {feedback_col_index}")

    # Extract feedback data
    for row in values[1:]: # Skip header row
```

Source-Code

```
        if len(row) > feedback_col_index and row[feedback_col_index].strip():
            feedback_list.append(row[feedback_col_index].strip())

    logger.info(f"Successfully extracted {len(feedback_list)} feedback entries")
    return feedback_list

except requests.exceptions.HTTPError as http_err:
    if response.status_code == 403:
        logger.error("Access denied. Check if the sheet is publicly accessible or API key
has proper permissions")
        raise ValueError("Access denied. Make sure the Google Sheet is publicly
accessible.")
    elif response.status_code == 404:
        logger.error("Spreadsheet not found")
        raise ValueError("Spreadsheet not found. Check the spreadsheet ID.")
    else:
        logger.error(f"HTTP error occurred: {http_err}")
        raise ValueError(f"Failed to fetch data from Google Sheets: {http_err}")
except Exception as e:
    logger.error(f"Error fetching feedback data: {e}")
    raise ValueError(f"Error accessing Google Sheets: {str(e)}")

def test_connection(self):
    """Test Google Sheets API connection"""
    try:
        # Test with a simple API call
        url = "https://sheets.googleapis.com/v4/spreadsheets"
        params = {'key': self.api_key}

        response = requests.get(url, params=params)
        return response.status_code == 200
    except Exception as e:
        logger.error(f"Connection test failed: {e}")
        return False

def get_sheet_info(self, spreadsheet_id):
    """Get basic information about the spreadsheet"""
    try:
        url = f"{self.base_url}/{spreadsheet_id}"
        params = {
            'key': self.api_key,
```

Source-Code

```
        'fields': 'properties.title,sheets.properties'
    }

    response = requests.get(url, params=params)
    response.raise_for_status()

    return response.json()
except Exception as e:
    logger.error(f"Error getting sheet info: {e}")
    return None
```

/backend/app

__init__.py

"""

Flask application factory

"""

from flask import Flask

from flask_cors import CORS

from dotenv import load_dotenv

from app.config import INSTANCE_PATH

def create_app():

Load environment variables

load_dotenv()

INSTANCE_PATH.mkdir(parents=True, exist_ok=True)

Create Flask app

app = Flask(__name__)

Enable CORS for all routes

CORS(app, origins=['http://localhost:3000', 'http://127.0.0.1:3000'])

Load configuration

from app.config import Config

app.config.from_object(Config)

Source-Code

```
from app.models import db
from app.db_setup import ensure_mysql_database_exists

if Config.uses_mysql():
    ensure_mysql_database_exists()
    app.config.setdefault(
        "SQLALCHEMY_ENGINE_OPTIONS",
        {"pool_pre_ping": True, "pool_recycle": 280},
    )

db.init_app(app)
with app.app_context():
    db.drop_all()
    db.create_all()

# Register blueprints/routes
from app.routes import api_bp
app.register_blueprint(api_bp, url_prefix='/api')

# Health check route
@app.route('/')
def index():
    return {
        'message': 'Feedback Analyzer Backend API',
        'status': 'running',
        'version': '1.0.0'
    }

return app
```

config.py

```
"""
Configuration settings for Feedback Analyzer
Optimized for Docker deployment
"""

import os
from pathlib import Path
from urllib.parse import quote_plus
from dotenv import load_dotenv
```

Source-Code

```
# Load environment variables from .env file
load_dotenv()

# Backend root (parent of app package) — used for SQLite instance folder
BACKEND_ROOT = Path(__file__).resolve().parent.parent
INSTANCE_PATH = BACKEND_ROOT / 'instance'

class Config:
    """Application configuration class"""

    # Flask Settings
    SECRET_KEY = os.getenv('SECRET_KEY', 'dev-secret-key-change-in-production')
    DEBUG = os.getenv('FLASK_DEBUG', 'False').lower() == 'true'
    FLASK_ENV = os.getenv('FLASK_ENV', 'development')

    # API Keys (Backward compatible with both naming conventions)
    GOOGLE_AI_API_KEY = os.getenv('GOOGLE_AI_API_KEY')
    GOOGLE_SHEETS_API_KEY = os.getenv('GOOGLE_SHEETS_API_KEY') or
os.getenv('GOOGLE_API_KEY')
    GOOGLE_API_KEY = GOOGLE_SHEETS_API_KEY

    # Google Sheets API Settings
    SHEETS_API_SCOPE = ['https://www.googleapis.com/auth/spreadsheets.readonly']

    # AI Model Settings
    AI_MODEL_NAME = os.getenv('AI_MODEL_NAME', 'gemini-2.5-flash')
    MAX_TOKENS = int(os.getenv('MAX_TOKENS', '1000'))

    # CORS Settings (important for Docker)
    CORS_ORIGINS = os.getenv('CORS_ORIGINS', '*').split(',')

    # Server Settings
    HOST = os.getenv('HOST', '0.0.0.0') # Important for Docker
    PORT = int(os.getenv('PORT', '8000'))

    # Logging
    LOG_LEVEL = os.getenv('LOG_LEVEL', 'INFO')

    # Database: set MYSQL_DATABASE (and credentials) for MySQL; else SQLite fallback.
    # Optional override: SQLALCHEMY_DATABASE_URI
```


Source-Code

```
SQLALCHEMY_TRACK_MODIFICATIONS = False
MYSQL_USER = os.getenv('MYSQL_USER', 'root')
MYSQL_PASSWORD = os.getenv('MYSQL_PASSWORD', '')
MYSQL_HOST = os.getenv('MYSQL_HOST', 'localhost')
MYSQL_PORT = os.getenv('MYSQL_PORT', '3306')
MYSQL_DATABASE = os.getenv('MYSQL_DATABASE', '').strip()

_default_sqlite = INSTANCE_PATH / 'saved_analyses.db'
_explicit_uri = os.getenv('SQLALCHEMY_DATABASE_URI', '').strip()

if _explicit_uri:
    SQLALCHEMY_DATABASE_URI = _explicit_uri
elif MYSQL_DATABASE:
    _auth = quote_plus(MYSQL_USER)
    _pw = quote_plus(MYSQL_PASSWORD) if MYSQL_PASSWORD else ""
    _cred = f"{_auth}:{_pw}" if MYSQL_PASSWORD else _auth
    SQLALCHEMY_DATABASE_URI = (
        f"mysql+pymysql://{_cred}@{MYSQL_HOST}:{MYSQL_PORT}/"
        f"{MYSQL_DATABASE}?charset=utf8mb4"
    )
else:
    SQLALCHEMY_DATABASE_URI = 'sqlite:/// ' + _default_sqlite.as_posix()

@classmethod
def uses_mysql(cls) -> bool:
    return 'mysql' in (cls.SQLALCHEMY_DATABASE_URI or '').lower()

@classmethod
def validate_config(cls):
    """Validate required configuration"""
    required_vars = {
        'GOOGLE_AI_API_KEY': cls.GOOGLE_AI_API_KEY,
        'GOOGLE_SHEETS_API_KEY': cls.GOOGLE_SHEETS_API_KEY
    }

    missing_vars = [name for name, value in required_vars.items() if not value]

    if missing_vars:
        raise ValueError(
            f"✗ Missing required environment variables: {', '.join(missing_vars)}\n"
            f"Please check your .env file or environment variables."
        )
```

Source-Code

```
)

return True

@classmethod
def get_config_summary(cls):
    """Get a summary of current configuration (safe for logging)"""
    return {
        'flask_env': cls.FLASK_ENV,
        'debug': cls.DEBUG,
        'host': cls.HOST,
        'port': cls.PORT,
        'ai_model': cls.AI_MODEL_NAME,
        'max_tokens': cls.MAX_TOKENS,
        'cors_origins': cls.CORS_ORIGINS,
        'google_ai_configured': bool(cls.GOOGLE_AI_API_KEY),
        'google_sheets_configured': bool(cls.GOOGLE_SHEETS_API_KEY),
        'database_backend': 'mysql' if cls.uses_mysql() else 'sqlite',
        'mysql_database': cls.MYSQL_DATABASE if cls.uses_mysql() else None,
    }
```

db_setup.py

```
"""
Ensure MySQL database exists before SQLAlchemy connects (avoids error 1049).
"""

from __future__ import annotations

import logging
import re

from sqlalchemy.engine.url import make_url

logger = logging.getLogger(__name__)

def _safe_db_name(name: str | None) -> str | None:
    if not name:
        return None
    name = str(name).strip()
    if not name or not re.match(r"^[a-zA-Z0-9_]+$", name):
```

Source-Code

```
        return None
    return name

def ensure_mysql_database_exists() -> None:
    """CREATE DATABASE IF NOT EXISTS when using MySQL (from URI or MYSQL_*)."""
    from app.config import Config

    if not Config.uses_mysql():
        return

    uri = Config.SQLALCHEMY_DATABASE_URI
    try:
        url = make_url(uri)
    except Exception as e:
        logger.error("Invalid SQLALCHEMY_DATABASE_URI / MySQL URL: %s", e)
        raise

    db_name = _safe_db_name(url.database)
    if not db_name:
        logger.warning(
            "Could not resolve MySQL database name from URL; "
            "create the database manually (e.g. CREATE DATABASE smart_feedback_ai)."
        )
        return

    try:
        import pymysql
    except ImportError:
        logger.error("pymysql is required for MySQL; pip install pymysql")
        raise

    host = url.host or "localhost"
    port = int(url.port or 3306)
    user = url.username or "root"
    password = url.password or ""

    try:
        conn = pymysql.connect(
            host=host,
            port=port,
```

Source-Code

```
        user=user,
        password=password,
        charset="utf8mb4",
    )
    try:
        with conn.cursor() as cur:
            cur.execute(
                f"CREATE DATABASE IF NOT EXISTS `{db_name}` "
                "CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci"
            )
        conn.commit()
        logger.info("MySQL database ready: %s", db_name)
    finally:
        conn.close()
    except Exception as e:
        logger.error(
            "Could not ensure MySQL database %r exists. "
            "Create it manually: CREATE DATABASE %s CHARACTER SET utf8mb4 COLLATE "
            "utf8mb4_unicode_ci; — %s",
            db_name,
            db_name,
            e,
        )
    Raise
```

routes.py

```
"""
```

API Routes - Updated for Hybrid Analysis

```
"""
```

```
from flask import Blueprint, request, jsonify
from app.services.sheets_service import GoogleSheetsService
from app.services.ai_service import FeedbackSummarizer
from app.config import Config
import logging
from datetime import datetime, timedelta, timezone

from flask import send_file, make_response
from app.services.export_service import ExportService
```

Source-Code

```
import uuid
import os
import json

from app.models import db, SavedAnalysis

# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def _isoformat_saved_at(dt):
    """Serialize DB datetimes as UTC ISO strings so browsers parse local time correctly."""
    if dt is None:
        return None
    if dt.tzinfo is not None:
        return dt.astimezone(timezone.utc).isoformat().replace('+00:00', 'Z')
    return dt.isoformat() + 'Z'

def mask_google_ai_api_key(key):
    """Non-reversible mask for audit display (never store raw keys)."""
    if not key:
        return 'N/A'
    k = str(key).strip()
    if len(k) <= 12:
        return '***'
    return f"{k[:8]}...{k[-4:]}"

api_bp = Blueprint('api', __name__)

# Initialize services
sheets_service = GoogleSheetsService()
summarizer = FeedbackSummarizer()

@api_bp.route('/health', methods=['GET'])
def health_check():
    """Health check endpoint"""
    try:
        Config.validate_config()
```

Source-Code

```
_s = Config.get_config_summary()
return jsonify({
    'status': 'healthy',
    'message': 'Hybrid Feedback Analyzer Backend is running',
    'timestamp': datetime.utcnow().isoformat(),
    'version': '2.0-hybrid',
    'database_backend': _s.get('database_backend'),
    'mysql_database': _s.get('mysql_database'),
}), 200

except Exception as e:
    logger.error(f"Health check failed: {e}")
    return jsonify({
        'status': 'unhealthy',
        'error': str(e)
    }), 500

@api_bp.route('/analyze-feedback', methods=['POST'])
def analyze_feedback():
    """
    Hybrid feedback analysis endpoint
    """
    try:
        # Validate request
        if not request.is_json:
            return jsonify({'error': 'Content-Type must be application/json'}), 400

        data = request.get_json()

        # Validate required parameters
        spreadsheet_id = data.get('spreadsheet_id')
        if not spreadsheet_id:
            return jsonify({'error': 'spreadsheet_id is required'}), 400

        range_name = data.get('range_name', 'Sheet1!A:B')

        logger.info(f"🚀 Starting hybrid analysis for spreadsheet: {spreadsheet_id}")

        # Fetch feedback from Google Sheets
        feedback_data = sheets_service.get_feedback_data(spreadsheet_id, range_name)
```

Source-Code

```
if not feedback_data:
    return jsonify({
        'error': 'No feedback data found in the specified sheet/range',
        'suggestions': [
            'Check if the spreadsheet is publicly accessible',
            'Verify the range contains feedback data'
        ]
    }), 400

logger.info(f"
```

Source-Code

```
except Exception as e:
    logger.error(f"Unexpected error: {e}", exc_info=True)
    return jsonify({
        'error': 'An unexpected error occurred',
        'details': str(e)
    }), 500
```

```
@api_bp.route('/test-connection', methods=['GET'])
def test_connection():
    """Test API connections"""
    try:
        sheets_test = sheets_service.test_connection()

        return jsonify({
            'google_sheets': 'success' if sheets_test else 'failed',
            'ai_service': 'ready',
            'timestamp': datetime.utcnow().isoformat()
        }), 200
```

```
except Exception as e:
    logger.error(f"Connection test failed: {e}")
    return jsonify({'error': str(e)}), 500
```

```
@api_bp.route('/export/pdf', methods=['POST'])
def export_pdf():
    """Export analysis results as PDF"""
    try:
        if not request.is_json:
            return jsonify({'error': 'Content-Type must be application/json'}), 400

        data = request.get_json()
        analysis_data = data.get('analysis_data')
        metadata = data.get('metadata', {})

        if not analysis_data:
            return jsonify({'error': 'analysis_data is required'}), 400
```


Source-Code

```
logger.info("📄 Generating PDF report...")

# Generate PDF
pdf_buffer = ExportService.generate_pdf_report(analysis_data, metadata)

# Generate filename
timestamp = datetime.utcnow().strftime('%Y%m%d_%H%M%S')
filename = f'feedback_analysis_{timestamp}.pdf'

return send_file(
    pdf_buffer,
    mimetype='application/pdf',
    as_attachment=True,
    download_name=filename
)

except Exception as e:
    logger.error(f"Error generating PDF: {e}", exc_info=True)
    return jsonify({'error': f'Error generating PDF: {str(e)}'}), 500

@api_bp.route('/export/excel', methods=['POST'])
def export_excel():
    """Export analysis results as Excel"""
    try:
        if not request.is_json:
            return jsonify({'error': 'Content-Type must be application/json'}), 400

        data = request.get_json()
        analysis_data = data.get('analysis_data')
        metadata = data.get('metadata', {})

        if not analysis_data:
            return jsonify({'error': 'analysis_data is required'}), 400

        logger.info("📊 Generating Excel report...")

        # Generate Excel
        excel_buffer = ExportService.generate_excel_report(analysis_data, metadata)

        # Generate filename
        timestamp = datetime.utcnow().strftime('%Y%m%d_%H%M%S')
```

Source-Code

```
filename = f'feedback_analysis_{timestamp}.xlsx'

return send_file(
    excel_buffer,
    mimetype='application/vnd.openxmlformats-officedocument.spreadsheetml.sheet',
    as_attachment=True,
    download_name=filename
)

except Exception as e:
    logger.error(f"Error generating Excel: {e}", exc_info=True)
    return jsonify({'error': f'Error generating Excel: {str(e)}'}, 500)

@api_bp.route('/saved-analyses', methods=['POST'])
def save_analysis():
    """Persist current analysis results with a user-provided dataset name."""
    try:
        if not request.is_json:
            return jsonify({'error': 'Content-Type must be application/json'}), 400

        data = request.get_json()
        dataset_name = (data.get('dataset_name') or "").strip()

        if not dataset_name:
            return jsonify({'error': 'dataset_name is required'}), 400

        row = SavedAnalysis(
            dataset_name=dataset_name,
            saved_at=datetime.now(timezone.utc),
            api_key=Config.GOOGLE_AI_API_KEY,
        )
        db.session.add(row)
        db.session.commit()

        logger.info(f"
```

Source-Code

```
        'dataset_name': row.dataset_name,  
        'saved_at': _isoformat_saved_at(row.saved_at),  
        'api_key': row.api_key,  
    }], 201
```

```
except Exception as e:  
    db.session.rollback()  
    logger.error(f"Error saving analysis: {e}", exc_info=True)  
    return jsonify({'error': f'Error saving analysis: {str(e)}'}), 500
```

```
@api_bp.route('/saved-analyses', methods=['GET'])  
def list_saved_analyses():  
    """List saved analyses."""  
    try:  
        rows = SavedAnalysis.query.order_by(SavedAnalysis.saved_at.desc()).all()  
        return jsonify([  
            {  
                'id': r.id,  
                'dataset_name': r.dataset_name,  
                'saved_at': _isoformat_saved_at(r.saved_at),  
                'api_key': r.api_key,  
            }  
            for r in rows  
        ]), 200  
    except Exception as e:  
        logger.error(f"Error listing saved analyses: {e}", exc_info=True)  
        return jsonify({'error': str(e)}), 500
```

```
@api_bp.route('/saved-analyses/<int:saved_id>', methods=['GET'])  
def get_saved_analysis(saved_id):  
    """Get a saved analysis record."""  
    try:  
        row = db.session.get(SavedAnalysis, saved_id)  
        if not row:  
            return jsonify({'error': 'Not found'}), 404  
  
        return jsonify({  
            'id': row.id,  
            'dataset_name': row.dataset_name,
```

Source-Code

```
        'saved_at': _isoformat_saved_at(row.saved_at),
        'api_key': row.api_key,
    }, 200
except Exception as e:
    logger.error(f"Error loading saved analysis: {e}", exc_info=True)
    return jsonify({'error': str(e)}), 500


@api_bp.route('/saved-analyses/<int:saved_id>', methods=['DELETE'])
def delete_saved_analysis(saved_id):
    """Remove a saved analysis row."""
    try:
        row = db.session.get(SavedAnalysis, saved_id)
        if not row:
            return jsonify({'error': 'Not found'}), 404

        db.session.delete(row)
        db.session.commit()
        logger.info(f"🗑 Deleted saved analysis id={saved_id}")
        return jsonify({'success': True, 'id': saved_id}), 200
    except Exception as e:
        db.session.rollback()
        logger.error(f"Error deleting saved analysis: {e}", exc_info=True)
        return jsonify({'error': str(e)}), 500
```

saved_analysis_fields.py

"""

Derive structured saved-analysis fields from analysis API payload.

"""

from __future__ import annotations

```
def overall_description_from_summary(summary: str | None, max_lines: int = 3,
max_chars: int = 1500) -> str:
    """Up to `max_lines` non-empty lines of the AI summary (overall description)."""
    text = (summary or "").strip()
    lines: list[str] = []
    for raw in text.split("\n"):
        line = raw.strip()
```

Source-Code

```
if line.startswith("##"):
    line = line.lstrip("#").strip()
if line:
    lines.append(line)
if len(lines) >= max_lines:
    break
result = "\n".join(lines[:max_lines])
if len(result) > max_chars:
    result = result[: max_chars - 1].rstrip() + "..."
return result
```

```
def recommendations_for_db(suggestions: list | None, max_points: int = 5) -> list[str]:
    """
    Store up to `max_points` (typical target 4–5) recommendation strings.
    Fewer if the model returned less. Each item may be multi-line; kept as one string.
    """
    out: list[str] = []
    for s in (suggestions or [])[:max_points]:
        t = str(s).strip()
        if t:
            out.append(t)
    return out
```

backend/query.py

```
from app import create_app
from app.models import SavedAnalysis
```

```
app = create_app()
```

```
with app.app_context():
    analyses = SavedAnalysis.query.all()
    if analyses:
        print("+---+-----+-----+-----+")
        print("| id | dataset_name | saved_at | api_key |")
        print("+---+-----+-----+-----+")
        for analysis in analyses:
            print(f"| {analysis.id:2} | {analysis.dataset_name:<17} | {analysis.saved_at} | {analysis.api_key} |")
```

Source-Code

```
print("+---+-----+-----+-----+")
else:
    print("+---+-----+-----+-----+")
    print("| id | dataset_name   | saved_at   | api_key   |")
```

backend/test_config.py

```
"""
Test configuration settings
"""

from app.config import Config
import json

def test_config():
    print("="*60)
    print("🔧 Testing Configuration")
    print("="*60)

    try:
        # Validate config
        Config.validate_config()
        print("✅ Configuration validation passed")

        # Print config summary
        summary = Config.get_config_summary()
        print("\n📄 Configuration Summary:")
        print(json.dumps(summary, indent=2))

        print("\n✅ All configuration tests passed!")
        return True

    except ValueError as e:
        print(f"\n❌ Configuration Error: {e}")
        return False

    except Exception as e:
        print(f"\n❌ Unexpected Error: {e}")
        return False

if __name__ == "__main__":
```